

# Shared CLI option parser + **AUTOLISP-...** installer

Specifications

clautolisp contributors

May 25, 2026

## Contents

|           |                                                  |          |
|-----------|--------------------------------------------------|----------|
| <b>1</b>  | <b>Introduction</b>                              | <b>1</b> |
| <b>2</b>  | <b>Goals</b>                                     | <b>1</b> |
| <b>3</b>  | <b>The option-spec datum</b>                     | <b>2</b> |
| <b>4</b>  | <b>Generic parser</b>                            | <b>2</b> |
| <b>5</b>  | <b>The CLI-OPTIONS struct</b>                    | <b>2</b> |
| <b>6</b>  | <b>Encoding registry</b>                         | <b>3</b> |
| <b>7</b>  | <b>Transmit-options installer</b>                | <b>3</b> |
| <b>8</b>  | <b>The <code>--list-encodings</code> printer</b> | <b>4</b> |
| <b>9</b>  | <b>Test coverage</b>                             | <b>4</b> |
| <b>10</b> | <b>Synchronisation</b>                           | <b>4</b> |

## 1 Introduction

The `clautolisp.autolisp-cli` system is the single source of truth for everything both `clautolisp(1)` and `alfe(1)` need to share at the command-line surface: option-spec datum, generic parser, `cli-options` struct, value

parsers (encoding / dialect / host / ...), transmit-options installer, encoding alias registry, and the `cli-usage-error` condition.

Before this system existed both tools carried independent copies of the same parsing logic, and the encoding-resolution table lived in two places. The split was painful at maintenance time; this system collapses it.

Audience: clautolisp developers adding or modifying CLI options on either tool, or extending the encoding registry.

## 2 Goals

- One canonical option-spec language usable by both tools.
- One canonical `cli-options` struct (union of clautolisp + alfe slots) so the transmit-options installer sees a stable shape.
- One canonical encoding alias registry with case-insensitive matching, line-terminator-suffix support, and impl-introspected typo rejection.
- One `cli-usage-error` condition class, re-exported through `alfe.error` for backwards compatibility with alfe's existing test harness.

## 3 The option-spec datum

```
(defstruct option-spec (longs nil :type list) ; ("load" "ld") (shorts nil :type list) ; ("l") (takes-arg-p nil :type boolean) (handler nil :type function)) ; (opts value name) -> void
```

A spec list is built by each tool from `*common-option-specs*` (the intersection: `--help`, `--version`, `--list-encodings`, `--dialect`, `--host`, `-l`, `-x`, `-i`, `-q`, `-v`, `-d`, `-e`, `-E`, `--no-init`, `--no-color`, plus the dialect short-hands) appended with tool-specific specs. First-match wins, so a tool can override a common spec by prepending its own.

## 4 Generic parser

`parse-arguments-with-spec SPECS ARGV &key INITIAL-OPTIONS` walks ARGV against SPECS, mutating an INITIAL-OPTIONS struct. Long options accept both `--opt VALUE` and `--opt=VALUE`. Unknown options signal `cli-usage-error`. Positional arguments push onto `cli-options-positional` and also queue a `(:file . PATH)` action so `clautolisp foo.lsp` works like `-l foo.lsp`.

## 5 The CLI-OPTIONS struct

A union of every slot both tools touch. Slots are tagged in comments with their relevance:

- AC — common to both
- A — alfe-only (clautolisp ignores)
- C — clautolisp-only (alfe ignores)

Some slots are simple flags (`help-p`, `version-p`, `list-encodings-p`); others carry keyword tokens (`dialect`, `host`, `mode`, `bootstrap-phase`), strings (`load-encoding`, `io-encoding`, `workdir`, `dwg`, `main`, `mock-input`, `gui`, `timeout`) or composite action lists (`actions`).

The `actions` slot is the shared parser's (`:KIND . PAYLOAD`) cons format. alfe post-translates it into `alfe.backend` action objects for its plan execution; the transmit-options installer (via `cli-options-transmit-bindings-for-alfe`) translates back to conses on the fly so the runtime-side renderer doesn't depend on alfe types.

## 6 Encoding registry

`*encoding-aliases*` is an alist of (CANONICAL KEYWORD ALIASES...) for the four mandatory encodings — US-ASCII, ISO-8859-1, WINDOWS-1252, UTF-8 — with the common spellings (`utf-8/utf8`, `latin-1/iso-8859-1/latin1`, `cp1252/windows-1252`, `ascii/us-ascii`).

`resolve-encoding-name` is the validator:

1. Strip a line-terminator suffix (`-mac / -dos / -unix / -lf / -cr / -crlf` — case-insensitive), recurse on the base, combine with (`BASE-KEYWORD :newline NEWLINE-KEYWORD`).
2. Match the (suffix-less) name against `*encoding-aliases*`.
3. For a plausible alphanumeric name not in the alias table, probe the impl's external-format registry (`sb-impl::get-external-format / ccl:lookup-character-encoding`). Accept if known.
4. Otherwise signal `cli-usage-error` mentioning the option.

`resolve-effective-encoding` adds the three-tier fallback for the transmit installer:

1. Explicit CLI value (already canonicalised).
2. Host locale ( $LC_{ALL} > LANG > LC_{CTYPE}$  via `clautolisp.autolisp-runtime:locale-default-`
3. Hard fallback `US-ASCII`.

Per encoding.issue, `*AUTOLISP-FILE-ENCODING*` and `*AUTOLISP-TERMINAL-ENCODING*` are never nil.

## 7 Transmit-options installer

`cli-options->transmit-bindings` `OPTIONS &key BACKEND VERSION-TEXT USAGE-TEXT` renders the 27-entry binding list for the `*AUTOLISP-...*` globals defined in `transmit-options.issue`, **AUTOLISP-VERSION** first per the issue’s remote-table convention.

`install-transmit-variables` `CONTEXT BINDINGS` interns each name and sets the binding-cell value via the runtime’s `set-variable`. Called once before the first user action runs (init files included).

`call-with-dynamic-transmit-binding` `CONTEXT NAME VALUE THUNK` scopes a dynamic `*AUTOLISP-LOAD-PATHNAME*` / `-EXPRESSION` / `-INTERACTIVE` for the duration of one action, reset on non-local exit.

## 8 The `-list-encodings` printer

`print-encodings` emits a two-section catalogue: the mandatory four with their CLI aliases, then every encoding the running CL implementation can hand to OPEN as `:external-format`. SBCL loads encoding tables on demand and lazy-loaded formats don’t register in `*external-formats*` until referenced, so `enumerate-implementation-encodings` probes a curated `*sbcl-common-encoding-probes*` list to surface the ISO-8859-\* / KOI8-\* / CP\* family.

The catalogue header announces the line-terminator suffix convention so a user discovers it without reading the spec.

## 9 Test coverage

Tests live in alfe’s `tests/cli-tests.lisp` and `backend-clautolisp-tests.lisp` because alfe is the heavier user of the surface; clautolisp’s own CLI tests are end-to-end via the binary.

Key checks:

- `cli-encoding-flags-stick` — canonicalisation lands the spelling per **encoding-aliases**.
- `cli-encoding-typo-rejected` — `-e uft-8` signals cli-usage-error rather than the cryptic external-format error it used to.
- `cli-encoding-line-terminator-suffix` — all six suffixes + case variations + base typos with valid suffix vs vice-versa.

## 10 Synchronisation

Per AGENTS.md (Documentation Synchronisation), changes to the code / this spec / the user-manual must be propagated. This subproject is a library and has no man page (the `clautolisp(1)` / `alfe(1)` man pages document the surface end-users see).